

Virtual Ray Sampling for Direct Volume Rendering using Hermite Interpolation

Balázs Csébfalvi*

Budapest University of Technology and Economics

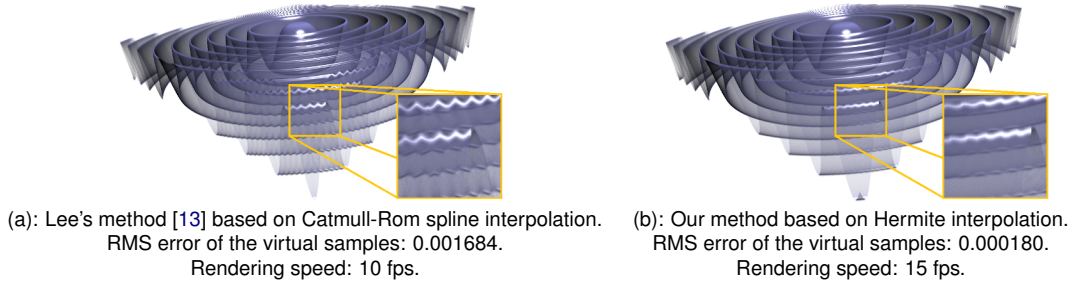


Figure 1: Virtual sampling using Catmull-Rom spline interpolation[13] (a) and the proposed Hermite interpolation (b). In both cases, 31 virtual samples are evaluated between each pair of consecutive true samples, which are obtained by an analytic evaluation of the Marschner-Lobb test signal [15]. Note that, using Hermite interpolation rather than Catmull-Rom spline interpolation, the rendering is 50% faster, the sampling artifacts are better reduced, and the RMS error of the virtual samples is an order of magnitude lower.

ABSTRACT

Virtual sampling has been proposed as an alternative of pre-integration to reduce artifacts caused by an insufficient sampling of the ray profiles in volume-rendering applications. According to the originally recommended implementation, a 3D tricubic B-spline reconstruction is applied to evaluate true samples along the rays, while between the true samples, virtual samples are calculated by using a 1D Catmull-Rom spline interpolation. The virtual samples can be evaluated much faster, but they can still well approximate the true samples. Therefore, the rendering performance can be drastically improved at the cost of a very slight quality degradation. Nevertheless, in this paper, we show that the original virtual sampling scheme does not exploit the fact, that the applied tricubic B-spline reconstruction filter together with its analytic derivative filters provide Hermite samples. Therefore, it is more natural to use Hermite interpolation along the rays rather than Catmull-Rom spline interpolation. From a sampling-theoretical point of view, this modification requires half of the true samples to guarantee the same reconstruction quality. Furthermore, concerning the evaluation of the true samples, we also investigate a recently published GPU-accelerated triquadratic B-spline filtering as an alternative of the tricubic B-spline filtering.

Index Terms: Direct volume rendering, reconstruction filtering, virtual sampling.

1 INTRODUCTION

Trilinear interpolation is a popular resampling technique used for GPU-accelerated direct volume rendering [1, 22, 11, 7], as it is hardwired in modern GPUs, and as such represents a reasonable compromise between rendering speed and image quality. Although higher-order filters, such as the tricubic B-spline [17, 15, 18, 19] or the Catmull-Rom spline [3, 10] were demonstrated to provide superior image quality, their brute-force implementations were significantly

slower. The first higher-order filtering technique optimized especially for the GPU was published by Sigg and Hadwiger [21] in 2005. They proposed to evaluate a tricubic B-spline filtering as a linear combination of eight trilinear samples rather than a linear combination of 64 nearest-neighbor samples, which reduces the number of the required texture fetches by a factor of eight. This method was also proposed to be further optimized [20] and adapted to a fast Catmull-Rom spline filtering [4]. Nevertheless, GPU-accelerated higher-order filtering techniques were mainly used for isosurface rendering, as in this case, the gradients have to be evaluated at most once for each pixel if a surface point is visible in the given pixel. In direct volume rendering, however, the gradients are also calculated for each sample along a ray, which requires the reconstruction of the three partial derivatives. For example, to calculate the analytic gradient of a tricubic B-spline reconstruction, for each partial derivative, eight additional trilinear texture fetches are necessary [21]. Therefore, the total number of texture fetches required for the density and gradient reconstructions is 32 for each sample position, which makes higher-order filtering too expensive for direct volume rendering.

2 RELATED WORK

In the literature, two different approaches have been proposed to reduce the cost of higher-order density and derivative reconstructions: (1) *combined filtering* and (2) *virtual sampling*.

2.1 Combined filtering

Previously, density and gradient reconstructions were handled as two separate computational tasks. For instance, in many practical volume-rendering applications, first the density is reconstructed by taking a single trilinear texture sample and afterwards the gradients are calculated from the central differences of six additional texture samples. Combined filtering techniques [5, 6], however, try to optimally use the same set of texture fetches for gradient estimation and for improving the approximation order of the density reconstruction as well. For example, the six additional trilinear texture fetches used for the gradient estimation were proposed to be reused for increasing the approximation order of the trilinear density reconstruction from two to three [5]. Recently, another com-

*e-mail: cseb@iit.bme.hu

binned filtering technique has been published, which evaluates a triquadratic B-spline reconstruction together with its analytic gradient from exactly the same eight trilinear texture samples [6]. Compared to the tricubic B-spline reconstruction [21], this approach reduces the number of texture fetches by a factor of four. Therefore, in this paper, we also evaluate the performance of this technique when combined with virtual sampling.

2.2 Virtual sampling

Virtual sampling [13] has been proposed as an alternative of pre-integration [16, 8, 14, 12] to reduce the artifacts introduced by an undersampling of the ray profiles. Between the true samples evaluated by a 3D tricubic B-spline filtering, virtual samples are calculated using a 1D Catmull-Rom spline interpolation along the viewing rays. As the virtual samples well approximate the true ones, and they are much faster to evaluate, a very good trade-off between image quality and rendering speed can be achieved. Nevertheless, we show that the Catmull-Rom spline interpolation does not fully exploit the information represented by the true tricubic samples. The exact gradients of the reconstructed function are evaluated anyway for each true sample applying the analytic derivatives of the tricubic B-spline filter [21]. Therefore, by projecting the gradients onto the normalized ray directions, we can easily obtain directional derivative samples along the rays. Thus, the true samples, in fact, represent Hermite data, and to interpolate between them, Hermite interpolation is a natural choice.

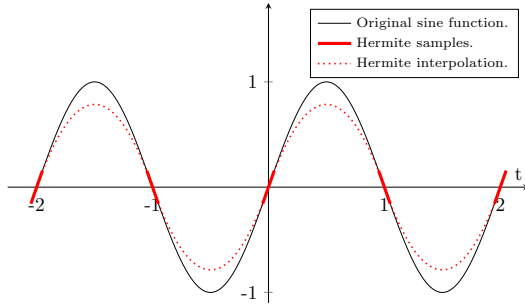


Figure 2: Reconstruction of a sine function using Hermite interpolation.

3 VIRTUAL SAMPLING USING HERMITE INTERPOLATION

In order to reconstruct function $f(t)$ from its samples $f_k = f(kT)$ and the corresponding derivatives $f'_k = f'(kT)$, the following ideal filtering can be applied [17]:

$$f(t) \approx \tilde{f}(t) = \sum_{k=-\infty}^{\infty} \left[f_k \varphi\left(\frac{t}{T} - k\right) + f'_k \psi\left(\frac{t}{T} - k\right) \right], \quad (1)$$

where the filters φ and ψ are defined as

$$\varphi(t) = \frac{\sin^2(\pi t)}{\pi^2 t^2}, \quad \text{and} \quad \psi(t) = \frac{\sin^2(\pi t)}{\pi^2 t}. \quad (2)$$

If $f(t)$ is band-limited and its band limit is B then, according to the generalization of the classical Shannon-Nyquist sampling theory, the sampling frequency $\nu = \frac{1}{T}$ should be greater than or equal to B to perfectly reconstruct $f(t)$. Note that, if the derivative samples were not available, twice as high sampling frequency would be necessary due to the Nyquist limit $\nu \geq 2B$. As the ideal reconstruction filters φ and ψ are of infinite extent, in practice, they are approximated by the following compact polynomial filters:

$$\varphi(t) \approx \varphi_H(t) = \begin{cases} 2|t|^3 - 3|t|^2 + 1 & \text{if } |t| \leq 1, \\ 0 & \text{otherwise.} \end{cases} \quad (3)$$

$$\psi(t) \approx \psi_H(t) = \begin{cases} t^3 - 2t|t| + t & \text{if } |t| \leq 1, \\ 0 & \text{otherwise.} \end{cases} \quad (4)$$

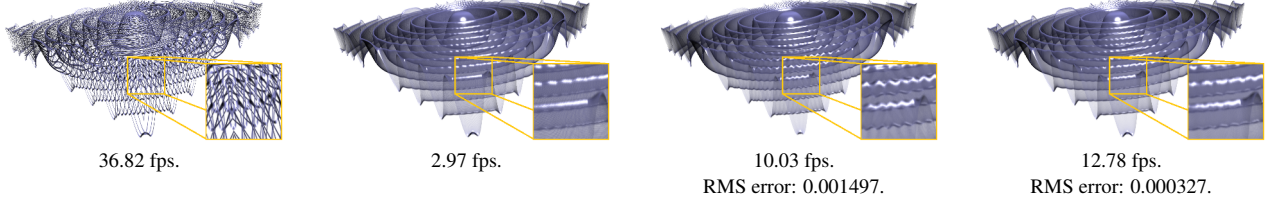
Filters $\varphi_H(t)$ and $\psi_H(t)$ result in the classical Hermite interpolation [17] that fits a third-degree polynomial $p_k(\tau)$ to the k^{th} pair of consecutive function samples f_k and f_{k+1} , and to the corresponding derivative samples f'_k and f'_{k+1} such that $p_k(0) = f_k$, $p_k(1) = f_{k+1}$, $p'_k(0) = f'_k$, and $p'_k(1) = f'_{k+1}$.

Note that Catmull-Rom spline interpolation recommended by Lee et al. [13] for virtual sampling is a special case of Hermite interpolation assuming that the derivative samples are not the exact derivatives of $f(t)$ but they are approximated by central differences: $f'_k \approx [f_{k+1} - f_{k-1}]/2T$. However, this approximation is risky as high-frequency components contained in the original ray profile might be suppressed or completely removed from the reconstructed ray profile. As an example, let us sample $f(t) = \sin(\pi t)$ at integer positions (see Fig. 2). Note that all the discrete samples $f_k = f(k)$ are equal to zero in this case. Therefore, the Catmull-Rom spline interpolation, as a special case of Hermite interpolation, would approximate the derivatives by the central differences of the discrete samples producing incorrect derivatives being equal to zero. Consequently, the reconstructed function would be a constant zero function. In contrast, if the exact derivatives are available at the sample positions then Hermite interpolation is able to reproduce the periodic nature of the sine function.

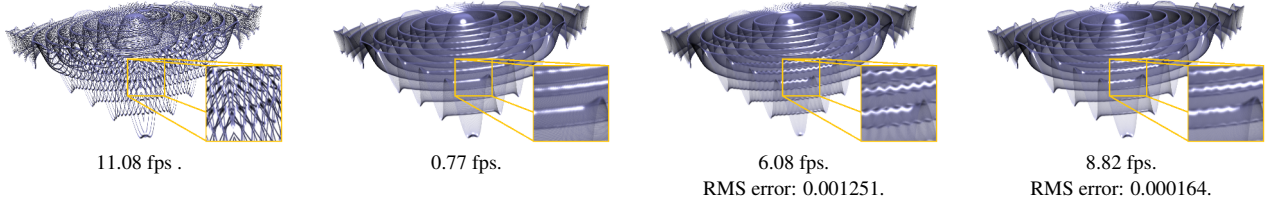
At first sight, this comparison might seem to be biased, as the extra information represented by the exact derivative samples gives an unfair advantage to Hermite interpolation. On the other hand, in the context of virtual sampling used for direct volume rendering, this extra information is, in fact, available for free. When the true samples are evaluated, the corresponding gradients are also calculated for the shading computations. Our key idea is to reuse the gradient information to improve virtual sampling in terms of image quality and computational efficiency as well. We assume that the applied gradient filter provides the analytic gradients of the reconstructed density function. For example, this criterion is fulfilled by the GPU-accelerated gradient estimation scheme proposed by Sigg and Hadwiger [21], as they use the analytic partial derivatives of the tricubic B-spline filter to compute the three partial derivatives the gradient consists of. Another example, is a recently published fast triquadratic B-spline reconstruction scheme [6], where the analytic partial derivatives are calculated from exactly the same eight trilinear samples that are used for the density reconstruction. Therefore, we test these two techniques for the evaluation of the true samples. As the analytic gradients are available for each true sample, it is easy to obtain the directional derivatives along the rays by simply projecting the gradients onto the given ray. This can be calculated as the dot product of the gradient and the normalized ray direction. As the true samples represent Hermite data, to interpolate between them, the classical Hermite interpolation can be used.

4 GPU IMPLEMENTATION

Using Hermite interpolation, the four coefficients of the third-degree polynomial $p_k(\tau)$, which approximates the ray profile between the true samples f_k and f_{k+1} , are considered to be the four unknowns of a linear equation system, while the four known values are f_k and f_{k+1} , and the corresponding directional derivatives f'_k and f'_{k+1} . In our GPU implementation, the known values are stored in a `vec4` variable `hermiteData`, while the inverse coefficient matrix of the underlying linear equation system is represented by the `mat4` variable `inverseMatrix`. The virtual samples are evaluated using the following GLSL code:



Using triquadratic B-spline interpolation [6] for evaluating the true samples.



Using tricubic B-spline interpolation [21] for evaluating the true samples.

- (a): No additional samples. (b): 15 additional true samples. (c): 31 additional virtual samples using Catmull-Rom interpolation. (d): 31 additional virtual samples using Hermite interpolation.

Figure 3: Virtual sampling between true samples evaluated by using a triquadratic B-spline interpolation [6] (top) and a tricubic B-spline interpolation [21] (bottom). (a): Undersampling the ray profiles without virtual sampling leading to an incorrect isosurface reconstruction. (b): Increasing the number of true samples by a factor of 16 drastically reducing the rendering performance. (c-d): Adding 31 virtual samples between each pair of consecutive true samples using Catmull-Rom spline interpolation ((c): Lee’s method [13]) and Hermite interpolation ((d): our method). Note that, using our method rather than Lee’s method, the rendering is faster, the sampling artifacts are better reduced, and the RMS error of the virtual samples is an order of magnitude lower.

```
vec4 hermiteData = vec4(
    previousSample.w,
    dot(previousSample.xyz, rayDir) * stepSize,
    currentSample.w,
    dot(currentSample.xyz, rayDir) * stepSize );

const mat4 inverseMatrix = mat4(
    1.0, 0.0, 0.0, 0.0,
    0.0, 1.0, 0.0, 0.0,
    -3.0, -2.0, 3.0, -1.0,
    2.0, 1.0, -2.0, 1.0 );

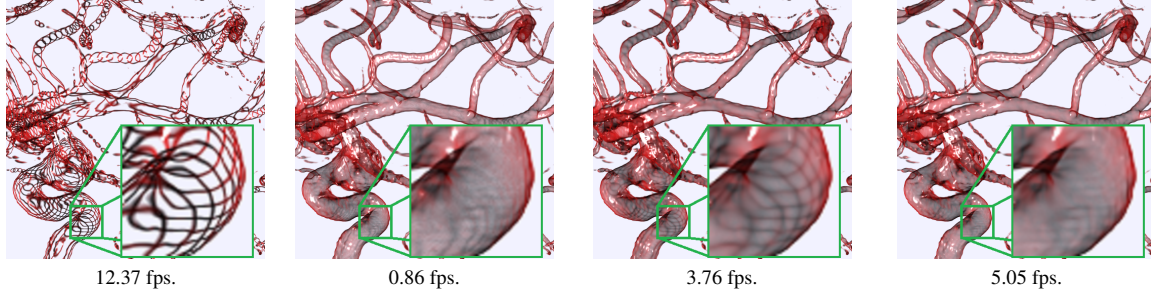
vec4 polynomial = hermiteData * inverseMatrix;
float tau = float(n) / numberOfVirtualSamplesPlusOne;
vec4 tauPowers =
    vec4(1.0, tau, tau * tau, tau * tau * tau);
float density = dot(polynomial, tauPowers);
```

The `vec4` variables `previousSample` and `currentSample` represent two consecutive true samples together with the corresponding analytic gradients. The first three components store the three partial derivatives the gradients consist of, while the fourth component is the density sample. These samples are evaluated by using either a triquadratic B-spline filtering [6] or a tricubic B-spline filtering [21]. The directional derivatives in variable `hermiteData` are obtained by projecting each gradient onto the normalized ray direction, which can be calculated as a dot product of the two vectors. The result of the dot product is further multiplied by variable `stepSize`, which is the sampling distance T defined in volume space. This multiplication is necessary, since the gradients are defined in volume space assuming that the distance between the neighboring voxels is one, while the derivatives of polynomial $p_k(\tau)$ are defined assuming that $\tau \in [0, 1]$. After having the Hermite

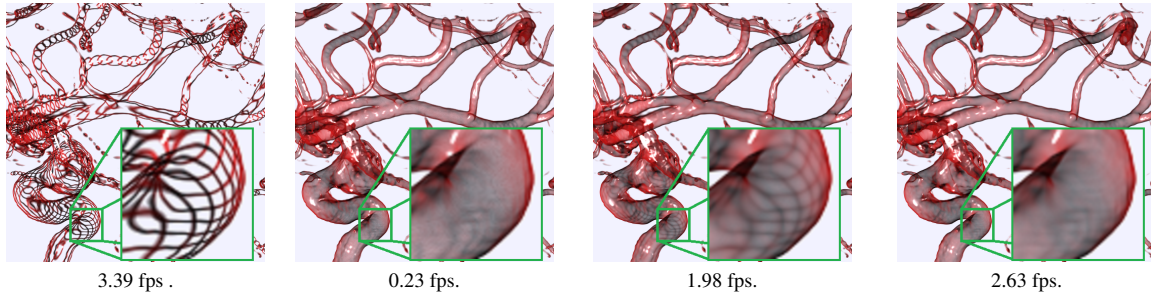
samples calculated, the polynomial coefficients for the interpolation are obtained by multiplying `hermiteData` with the pre-calculated inverse coefficient matrix `inverseMatrix`. Once the polynomial coefficients stored in variable `polynomial` are available, an arbitrary number of virtual samples can be evaluated between each pair of consecutive true samples. During virtual sampling, parameter `tau` is between zero and one, and calculated from `n`, which is the serial number of the given virtual sample.

5 HERMITE INTERPOLATION VERSUS CATMULL-ROM SPLINE INTERPOLATION

In order to evaluate the true samples, we implemented both triquadratic B-spline filtering [6] and tricubic B-spline filtering [21] on the GPU. Fig. 3 shows the comparison of these techniques when combined with virtual sampling using either Catmull-Rom spline interpolation (Lee’s method [13]) or the proposed Hermite interpolation. Intentionally, we applied a transfer function defined by a narrow peak emphasizing a single semitransparent isosurface. If virtual samples are not evaluated and the sampling distance is the same as the distance between the voxels then the ray profiles are undersampled. Thus, the isosurface cannot be reconstructed (see Fig. 3a). When the number of true samples is increased by a factor of 16, the isosurface is properly reconstructed, but the rendering performance is drastically reduced (see Fig. 3b). This is especially the case for the tricubic B-spline filtering, as here the total number of trilinear texture fetches necessary for the density and gradient reconstructions is 32, which makes this technique very expensive computationally. As a triquadratic B-spline filtering requires only eight texture fetches for reconstructing both the density and the gradient, it is approximately four times faster than a tricubic B-spline filtering. Nevertheless, both techniques are relatively slow if they



Using triquadratic B-spline interpolation [6] for evaluating the true samples.



Using tricubic B-spline interpolation [21] for evaluating the true samples.

(a): No additional samples. (b): 15 additional true samples. (c): 31 additional virtual samples using Catmull-Rom interpolation. (d): 31 additional virtual samples using Hermite interpolation.

Figure 4: Virtual sampling between true samples evaluated by using a triquadratic B-spline interpolation [6] (top) and a tricubic B-spline interpolation [21] (bottom). (a): Undersampling the ray profiles without virtual sampling leading to an incorrect isosurface reconstruction. (b): Increasing the number of true samples by a factor of 16 drastically reducing the rendering performance. (c-d): Adding 31 virtual samples between each pair of consecutive true samples using Catmull-Rom spline interpolation ((c): Lee’s method [13]) and Hermite interpolation ((d): our method). Note that, using our method rather than Lee’s method, both the rendering performance and the image quality can be improved.

are not combined with virtual sampling. However, by evaluating 31 additional virtual samples rather than 15 additional true samples, approximately the same image quality can be achieved at a significantly lower cost (see Fig. 3c and Fig. 3d). Increasing the number of virtual samples, the bottleneck is not the cost of the true sampling anymore, but the complexity of the virtual sampling. Note that, due to its simplicity, the proposed Hermite interpolation is around 30-40% faster than the previously recommended Catmull-Rom spline interpolation [13]. Moreover, because of its sampling-theoretical advantages discussed in Section 3, Hermite interpolation more efficiently reduces the sampling artifacts (see Fig. 3d compared to Fig. 3c). This is also demonstrated by Fig. 1, where the true samples are obtained by an analytic evaluation of the Marschner-Lobb test signal [15]. Note that, in this case, Hermite interpolation is even 50% faster than Catmull-Rom spline interpolation. We also tested virtual sampling on a CT scan of an aneurysm. Using Hermite interpolation rather than Catmull-Rom spline interpolation, the sampling artifacts are significantly reduced, and additionally, more high-frequency details, such as the thin blood vessels, can be better reproduced (see Fig. 4). Furthermore, due to the simpler implementation of virtual sampling, the rendering is 30-40% faster just as for the Marschner-Lobb test using practical filters for the evaluation of the true samples.

6 CONCLUSION AND FUTURE WORK

In this paper, we have shown that Hermite interpolation is more efficient to use for virtual ray sampling than the originally recommended Catmull-Rom spline interpolation [13]. If for each true sample the analytic gradients are also evaluated by using the analytic derivatives of the reconstruction filter [21, 6] then Hermite samples along the rays can be easily evaluated, where the additional derivative samples are calculated by projecting the gradients onto the normalized ray direction. Therefore, Hermite interpolation can be naturally used between the true samples. We have shown that this approach can potentially reproduce high-frequency details that are suppressed or removed by the Catmull-Rom spline interpolation. Moreover, due to its simplicity, Hermite interpolation is much faster to evaluate.

As the Hermite samples provide a piecewise polynomial representation of the ray profiles, in our future work, we plan to combine Hermite interpolation with adaptive numerical integration techniques [9, 2] in order to reduce the number of the necessary virtual samples between each pair of consecutive true samples.

ACKNOWLEDGMENTS

This work has been supported by the Hungarian Scientific Research Fund (OTKA K-145970).

REFERENCES

- [1] B. Cabral, N. Cam, and J. Foran. Accelerated volume rendering and tomographic reconstruction using texture mapping hardware. In *Proc. IEEE Symposium on Volume Visualization*, pp. 91–98, 1994. doi: 10.1145/197938.197972 1
- [2] L. Campagnolo, W. Celes, and L. Figueiredo. Accurate volume rendering based on adaptive numerical integration. In *Proc. Conference on Graphics, Patterns and Images (SIBGRAPI)*, pp. 17–24, 2015. doi: 10.1109/SIBGRAPI.2015.27 4
- [3] E. Catmull and R. Rom. A class of local interpolating splines. *Computer Aided Geometric Design*, pp. 317–326, 1974. doi: 10.1016/B978-0-12-079050-0.50020-5 1
- [4] B. Csébfalvi. Fast Catmull-Rom spline interpolation for high-quality texture sampling. *Computer Graphics Forum (Proc. EUROGRAPH-ICS)*, 37(2):455–462, 2018. doi: 10.1111/cgf.13375 1
- [5] B. Csébfalvi. Beyond trilinear interpolation: Higher quality for free. *ACM Transactions on Graphics (Proc. SIGGRAPH)*, 38(4), 2019. doi: 10.1145/3306346.3323032 1
- [6] B. Csébfalvi. One step further beyond trilinear interpolation and central differences: Triquadratic reconstruction and its analytic derivatives at the cost of one additional texture fetch. *Computer Graphics Forum (Proc. EUROGRAPHICS)*, 42(2):191–200, 2023. doi: 10.1111/cgf.14753 1, 2, 3, 4
- [7] K. Engel, M. Hadwiger, J. Kniss, C. Reck-Salama, and D. Weiskopf. *Real-Time Volume Graphics*. AK Peters Ltd., 2006. 1
- [8] K. Engel, M. Kraus, and T. Ertl. High-quality pre-integrated volume rendering using hardware-accelerated pixel shading. In *Proc. ACM SIGGRAPH/EUROGRAPHICS workshop on Graphics hardware*, pp. 9–16, 2001. doi: 10.1145/383507.383515 2
- [9] T. Etienne, D. Jönsson, T. Ropinski, C. Scheidegger, J. L. D. Comba, L. G. Nonato, R. M. Kirby, A. Ynnerman, and C. T. Silva. Verifying volume rendering using discretization error analysis. *IEEE Transactions on Visualization and Computer Graphics*, 20(1):140–154, 2014. doi: 10.1109/TVCG.2013.90 4
- [10] R. G. Keys. Cubic convolution interpolation for digital image processing. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, ASSP-29(6):1153–1160, 1981. doi: 10.1109/TASSP.1981.1163711 1
- [11] J. Krüger and R. Westermann. Acceleration techniques for GPU-based volume rendering. In *Proc. IEEE Visualization*, pp. 38–45, 2003. doi: 10.1109/VISUAL.2003.1250384 1
- [12] H. Kye, B. Shin, and Y. Shin. Interactive classification for pre-integrated volume rendering of high-precision volume data. *Graphical Models*, 70(6):125–132, 2008. doi: 10.1016/j.gmod.2008.05.001 2
- [13] B. Lee, J. Yun, J. Seo, B. Shim, Y. Shin, and B. Kim. Fast high-quality volume ray casting with virtual samplings. *IEEE Transactions on Visualization and Computer Graphics*, 16(6):1525–1532, 2010. doi: 10.1109/TVCG.2010.155 1, 2, 3, 4
- [14] E. Lum, B. Wilson, and K. L. Ma. High-quality lighting and efficient pre-integration for volume rendering. In *Proc. VisSym'04*, pp. 25–34, 2004. doi: 10.2312/VisSym/VisSym04/025-034 2
- [15] S. Marschner and R. Lobb. An evaluation of reconstruction filters for volume rendering. In *Proc. IEEE Visualization*, pp. 100–107, 1994. doi: 10.1109/VISUAL.1994.346331 1, 4
- [16] N. Max, P. Hanrahan, and R. Crawfis. Area and volume coherence for efficient visualization of 3D scalar functions. In *Proc. Workshop on Visualization*, pp. 27–33, 1990. doi: 10.1145/99307.99315 2
- [17] D. Mitchell and A. Netravali. Reconstruction filters in computer graphics. In *Proc. SIGGRAPH*, pp. 221–228, 1988. doi: 10.1145/54852.378514 1, 2
- [18] T. Möller, R. Machiraju, K. Mueller, and R. Yagel. Evaluation and design of filters using a Taylor series expansion. *IEEE Transactions on Visualization and Computer Graphics*, 3(2):184–199, 1997. doi: 10.1109/2945.597800 1
- [19] T. Möller, K. Mueller, Y. Kurzion, R. Machiraju, and R. Yagel. Design of accurate and smooth filters for function and derivative reconstruction. In *Proc. IEEE Symposium on Volume Visualization*, pp. 143–151, 1998. doi: 10.1145/288126.288189 1
- [20] D. Ruijters, B. M. ter Haar Romeny, and P. Suetens. Efficient GPU-based texture interpolation using uniform B-splines. *Journal of Graphics Tools*, 13(4):61–69, 2008. doi: 10.1080/2151237X.2008.10129269 1
- [21] C. Sigg and M. Hadwiger. Fast third-order texture filtering. In *GPU Gems 2: Programming Techniques for High-Performance Graphics and General-Purpose Computation*, pp. 313–329. Matt Pharr (ed.), Addison-Wesley, 2005. 1, 2, 3, 4
- [22] R. Westermann and T. Ertl. Efficiently using graphics hardware in volume rendering applications. In *Proc. SIGGRAPH*, pp. 169–177, 1998. doi: 10.1145/280814.280860 1